

RAG Document Pipeline

Outcome: A retrieval-augmented generation system for an internal knowledge base — ingest, chunk, embed, index, retrieve, and generate — with golden-set evaluation (hit@k 1.0, keyword recall 1.0, faithfulness-lite 0.845), FastAPI serving, HTTP document pull, and result sinks.

1. Why

Operations and support teams lose time searching across policies, runbooks, and FAQs. A retrieval-augmented pipeline turns those documents into a queryable index with grounded answers and measurable retrieval quality. This system exists to deliver that capability with local embeddings (no mandatory cloud LLM), optional OpenAI-compatible generation, CLI and HTTP serving, scheduled rebuild via Airflow, and published evaluation scores.

2. What — Deliverables

- **Ingest** → **chunk** → **embed** → **index** pipeline over Markdown, TXT, PDF, and JSON.
- **Knowledge corpus** — access control, incident response, billing FAQ, data retention, on-call handoff.
- **API source** — `pull_remote_docs` fetches configured HTTP documents (e.g. GitHub raw Markdown, JSON posts) into `data/remote_docs/` and merges at ingest.
- **Retrieval & generation** — top-k cosine search; offline extractive answerer or OpenAI-compatible LLM mode.
- **Evaluation** — golden Q&A; set with hit@k, keyword_recall, context_relevance, faithfulness_lite → `eval/results.json`.
- **Serving** — CLI `ask/run`; FastAPI `POST /ask` and `GET /health`.
- **API sinks** — push ask/eval payloads to local FastAPI landing and JSONPlaceholder.
- **Orchestration** — Airflow DAG for weekly rebuild + eval.

3. When — Delivery Plan

Phase	Focus	Typical duration
Design	Corpus inventory, chunk/embed config, eval golden set, API contracts	2–4 days
Build	Pipeline stages, index backends, FastAPI, remote-doc pull, sinks, Airflow DAG	1.5–2.5 weeks
Validate	Golden-set eval gates, ask smoke tests, sink receipts, offline fallback	3–4 days
Handoff	Config guide, ops runbook, acceptance on hit@k / faithfulness targets	2–3 days

Comparable scoped delivery: roughly **3–5 weeks**, longer if PDF-heavy corpora, SSO-backed LLM gateways, or multi-tenant index isolation are required.

4. How — Architecture & Stack

Default index backend is NumPy cosine similarity; Chroma and FAISS are optional. Embeddings use *sentence-transformers/all-MiniLM-L6-v2* (384-d) on CPU. Chunking uses overlapping character windows (size 500, overlap 80, min 40 chars). Generation defaults to extractive ranking of retrieved sentences; set *generate.mode: openai_compatible* with env-based credentials for LLM answers.

Concern	Choice
Embed	sentence-transformers/all-MiniLM-L6-v2 · 384 dims · batch 32
Index	NumPy cosine (default); Chroma / FAISS optional
Retrieve	top_k=4 · cosine distance
Generate	extractive (default) or openai_compatible
Eval	hit@k, keyword_recall, context_relevance, faithfulness_lite
Serve	CLI + FastAPI /ask · /health
API source	HTTP documents listed in config/pipeline.yaml
API sinks	Local FastAPI /ingest + JSONPlaceholder /posts
Orchestration	Airflow weekly rebuild + eval

5. Architecture Diagram

RAG Document Pipeline — Architecture

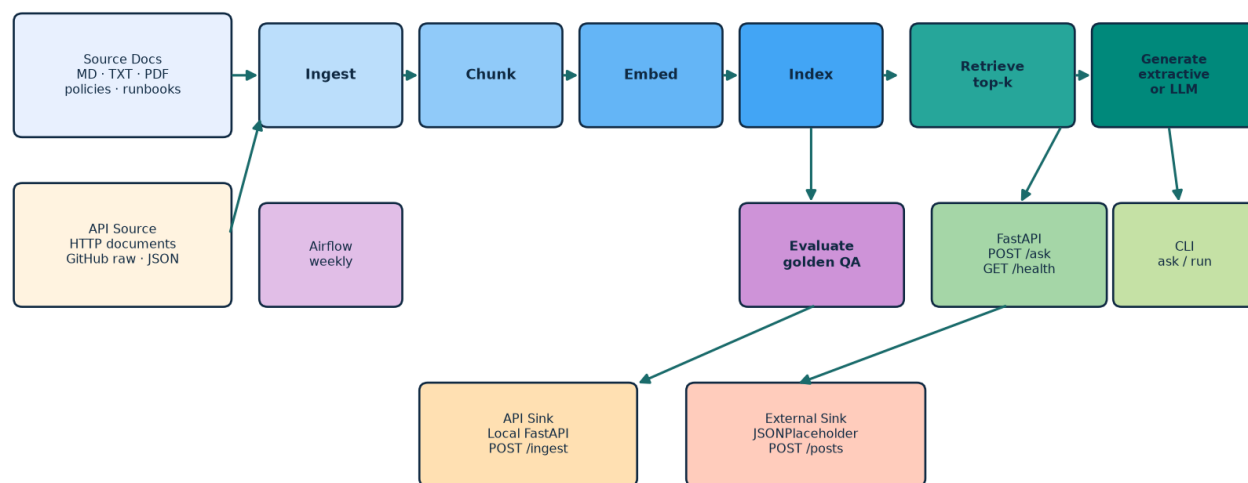


Figure 1. Document sources → RAG stages → evaluate / serve → HTTP sinks.

6. Technical Depth

Index & embedding metadata

Artifact	Value
Index backend	numpy
Indexed chunks	20
Embedding dimensions	384
Distance metric	cosine
Remote docs fetched	2 (GitHub flow MD + JSON post); ok_count=2
Corpus topics	Access control, incident response, billing FAQ, retention, on-call

Golden-set evaluation summary (8 questions, top_k=4)

Metric	Score
hit@k	1.00
keyword_recall	1.00
context_relevance	0.350
faithfulness_lite	0.845

All eight golden questions retrieved the expected document within top-k. Example coverage: quarterly access reviews for production apps; SEV1 external status-page timing; subscription downgrade effective dates; application log retention; SEV1 on-call acknowledge window; payment-failure grace period; contractor account expiry; legal-hold ownership.

7. Outcomes — What the Client Gets

- A complete RAG pipeline from documents to grounded answers with published eval scores.
- Offline-capable embeddings and extractive generation; optional LLM upgrade path.
- FastAPI and CLI interfaces for product and ops integration.
- HTTP remote-document ingest and dual result sinks for integration testing.
- Airflow DAG for scheduled index rebuild and evaluation.
- Config-driven chunk/embed/retrieve/generate settings without hard-coded secrets.

8. Repository

<https://github.com/user-JB007/rag-document-pipeline>

License: MIT. See *docs/architecture.md*, *config/pipeline.yaml*, and *eval/results.json* for design and measured quality.